Universidad de Granada

Periféricos y Dispositivos de Interfaz Humana
Curso 2025/2026

Practica 1. Entrada/Salida utilizando interrupciones con lenguaje C

Autora:
Inés Ruiz Sánchez

11 de marzo de 2026

UNIVERSIDAD
DE GRANADA

Índice

1. Introducción **2**

2. Requisitos mínimos **2**
2.1. Ejercicio 1: *gotoxy()* . . . 2
2.2. Ejercicio 2: *setcursortype()* . . . 3
2.3. Ejercicio 3: *setvideomode()* . . . 6
2.4. Ejercicio 4: *getvideomode()* . . . 7
2.5. Ejercicio 5: *textcolor()* . . . 7
2.6. Ejercicio 6: *textbackground()* . . . 8
2.7. Ejercicio 7: *clrscr()* . . . 9
2.8. Ejercicio 8: *cputchar()* . . . 12
2.9. Ejercicio 9: *getche()* . . . 13
2.10. Ejercicio 10: *pixel()* . . . 14

3. Requisitos ampliados **16**
3.1. Ejercicio 11 . . . 16
3.2. Ejercicio 12 . . . 16
3.3. Ejercicio 13 . . . 16

4. Anexo **16**
4.1. Ejemplo completo para la función *gotoxy()* . . . 16
4.2. Ejemplo completo para la función *setcursortype()* . . . 17
4.3. Ejemplo completo para la función *setvideomode()* . . . 18
4.4. Ejemplo completo para la función *getvideomode()* . . . 18
4.5. Ejemplo completo para las funciones *textcolor()*, *textbackground()* y *cputchar()* . . . 19
4.6. Ejemplo completo para la función *clrscr()* . . . 20
4.7. Ejemplo completo para la función *getche* . . . 21
4.8. Ejemplo completo para la función *pixel()* . . . 22
4.9. Código completo para el ejercicio 11 . . . 23
4.10. Código completo para el ejercicio 12 . . . 23
4.11. Código completo para el ejercicio 13 . . . 23

Índice de figuras

1. Ejecución de gotoxy() . . . 3
2. Ejecución de cursor normal (tipo 1) . . . 5
3. Ejecución de cursor grueso (tipo 2) . . . 5
4. Ejecución de cursor invisible (tipo 0) . . . 6
5. Ejemplo de color de texto y de fondo con palabra 'HOLA' . . . 9
6. Ejemplo de compilación y ejecución de clrscr() . . . 11
7. Ejemplo con la pantalla azul . . . 11
8. Ejemplo de pantalla 'limpia' . . . 12
9. Ejemplo de color de texto y de fondo con palabra 'PDIH' . . . 13
10. Ejemplo de getche() . . . 14
11. Ejemplo de la función pixel . . . 16

1. Introducción

Usando las subfunciones expuestas en el guión de la *Práctica 1*, hay que implementar en C/C++ un conjunto de funciones similares a las que ofrece la biblioteca *conio.lib* (*Console Input Output Library*), que permiten realizar tareas tales como cambiar la posición del cursor, borrar la pantalla, cambiar el color del texto, etc.

Nota: Las tildes en los comentarios de los ficheros importados con extensión *.c* no se reconocen en LATEX por lo que el carácter no se escribe. En el *Anexo* se han importado los ficheros al completo y es por eso por lo que o bien no aparece un caracter o bien el caracter no tiene tilde, pareciendo una falta ortográfica.

2. Requisitos mínimos

Realizar las siguientes 10 funciones, que deben diseñarse e implementarse de forma que se puedan reutilizar fácilmente en otros programas. El programa debe utilizar todas esas funciones para comprobar su correcto funcionamiento.

Hay una función muy usada en todos (o casi todos) los programas para hacer una pausa[1]. Esta función en proporcionada por el profesor y es la siguiente:

```c
void mi_pausa(){
    union REGS inregs, outregs;
    inregs.h.ah = 8;
    int86(0x21, &inregs, &outregs);
}
```

Listing 1: Función pausa

Además, para todos los programas se usan las siguientes dos cabeceras[2]:

```c
#include <stdio.h>
#include <dos.h>
```

Listing 2: Cabeceras

2.1. Ejercicio 1: *gotoxy()*

Objetivo: **colocar el cursor en una posición determinada**.

Para colocar el cursor en una posición determinada se usa la entrada $AH = 2$. Además, es necesario usar:

- DH: número de la fila
- DL: número de columna
- BH = 0

```c
void gotoxy(int x, int y){
    union REGS inregs, outregs;
    inregs.h.ah = 0x02;
    inregs.h.bh = 0x00;
    inregs.h.dh = y;
    inregs.h.dl = x;
    int86(0x10,&inregs,&outregs);
    return;
}
```

Listing 3: Función gotoxy()

Esta función[3] es proporcionada por el profesor, pero para hacerla más dinámica en el *main* se solicita al usuario las coordenadas x e y.

```c
int main(){
    int pos_x, pos_y; //Guardar posicion determinada

    // Solicitar coordenadas
    printf("Introduce la posicion x (columna)");     scanf("%d", &pos_x);
    printf("Introduce la posicion y (fila)");        scanf("%d", &pos_y);

    gotoxy(pos_x,pos_y); // Llamada a la funcion

    printf("*"); // Imprimir asterisco en la posicion

    mi_pausa();

    return 0;
}
```

Listing 4: Función principal para gotoxy()

Para ver el programa completo para probar el correcto funcionamiento de la función *gotoxy()* leer en el anexo 4.1.

Ejecución [1] para $x = 3$ e $y = 3$:

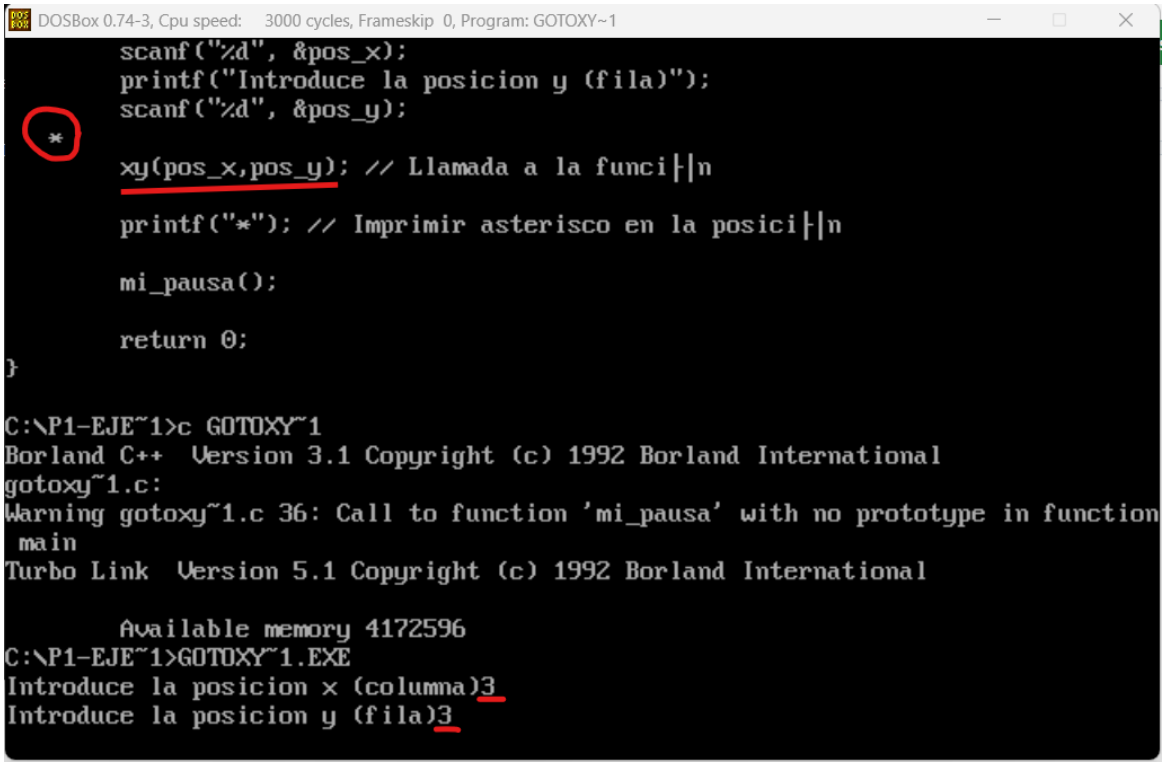

Figura 1: Ejecución de gotoxy()

2.2. Ejercicio 2: *setcursortype()*

Objetivo: **fijar el aspecto del cursor, debe admitir tres valores: *INVISIBLE*, *NORMAL* y *GRUESO*.**

Es una función proporcionada por el profesor, pero se ha cambiado el *main* para que el usuario pueda elegir el tipo de cursor por teclado.

```c
void setcursortype(int tipo_cursor){
    union REGS inregs, outregs;
    inregs.h.ah = 0x01;
    switch(tipo_cursor){
```

```c
        case 0: //invisible
            inregs.h.ch = 010;
            inregs.h.cl = 000;
            break;
        case 1: //normal
            inregs.h.ch = 010;
            inregs.h.cl = 010;
            break;
        case 2: //grueso
            inregs.h.ch = 000;
            inregs.h.cl = 010;
            break;

    }
    int86(0x10, &inregs, &outregs);
}
```

Listing 5: Función setcursortype()

Se ha cambiado el main para que el usuario pueda elegir el tipo de cursor por teclado:

```c
int main(){
    int tmp;

    printf("\nTipos de cursor. \n");
    printf("\tTipo 0: invisible \tTipo 1: normal \tTipo 2: grueso \n\nIntroduce solo el numero: ");
    scanf("%d", &tmp); // Leer de teclado

    printf("\nHas elegido tipo: %d\n", tmp); //Mostrar eleccion

    setcursortype(tmp);
    mi_pausa();

    return 0;
}
```

Listing 6: Función principal para setcursortype()

Para ver el programa completo para probar el correcto funcionamiento de la función *setcursortype()* leer en el anexo 4.2.

Ejecución para cursor normal [2]:

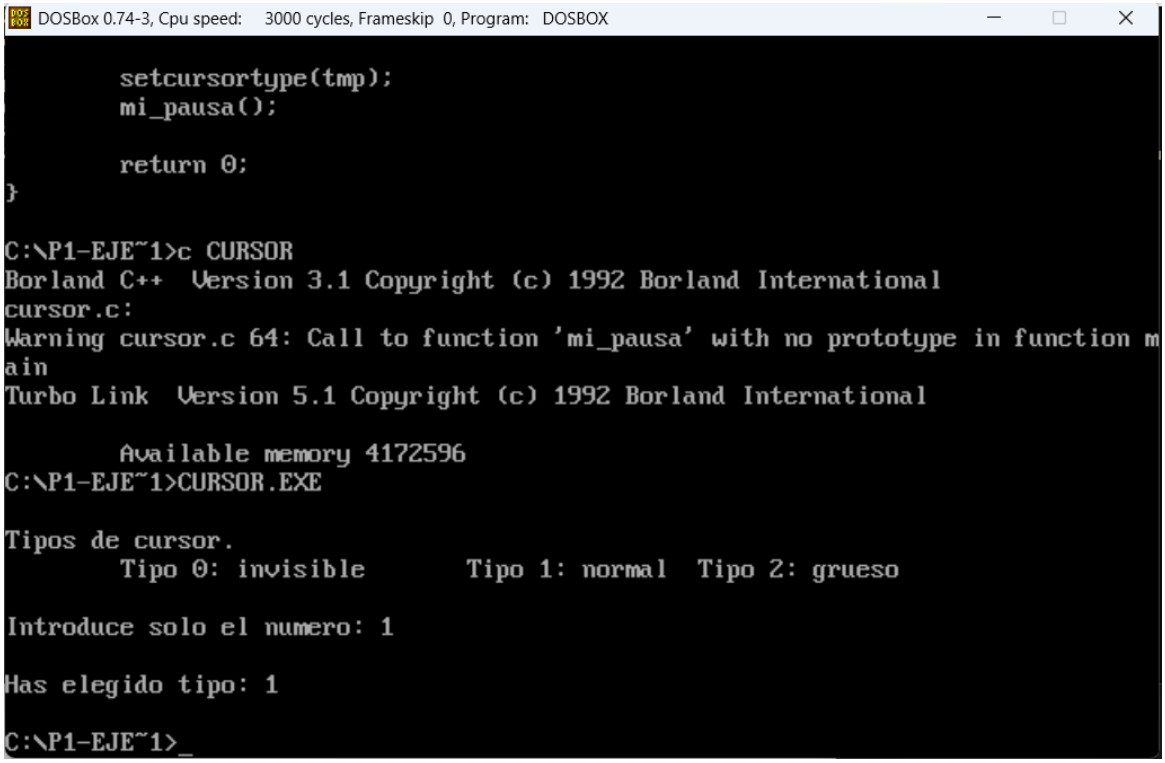

Figura 2: Ejecución de cursor normal (tipo 1)

Ejecución para cursor grueso [3]:

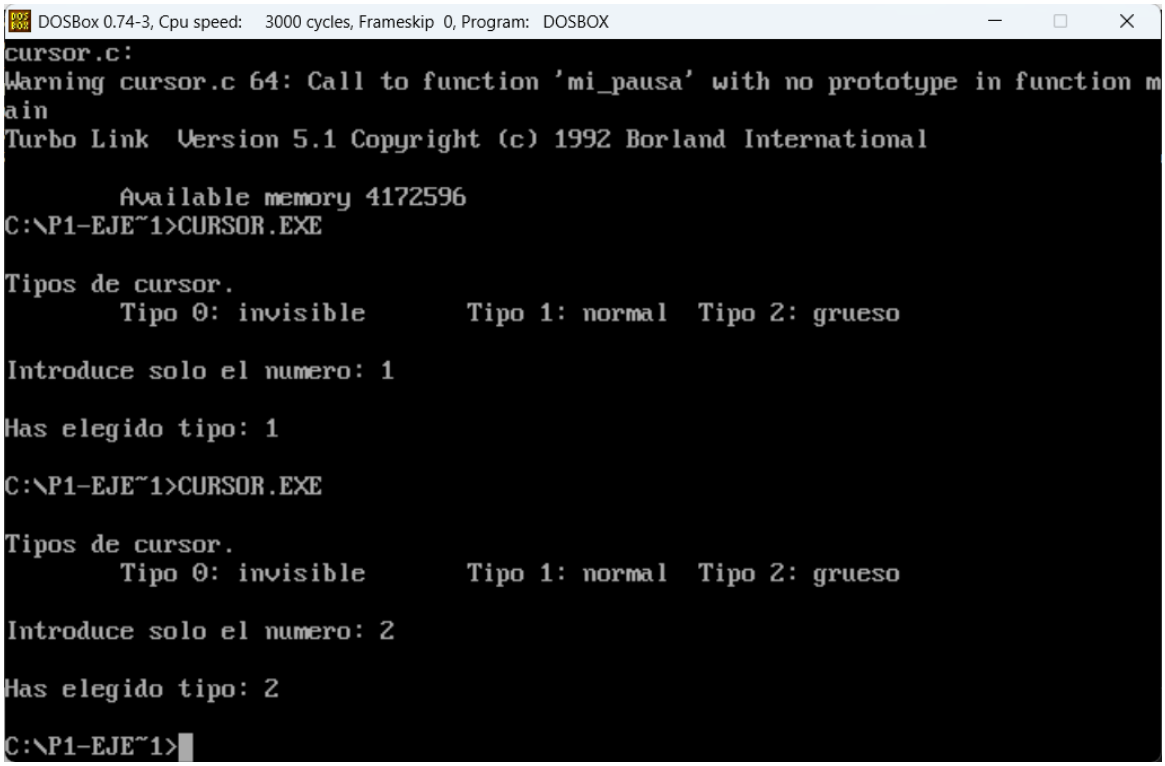

Figura 3: Ejecución de cursor grueso (tipo 2)

Ejecución para cursor invisible [4]:

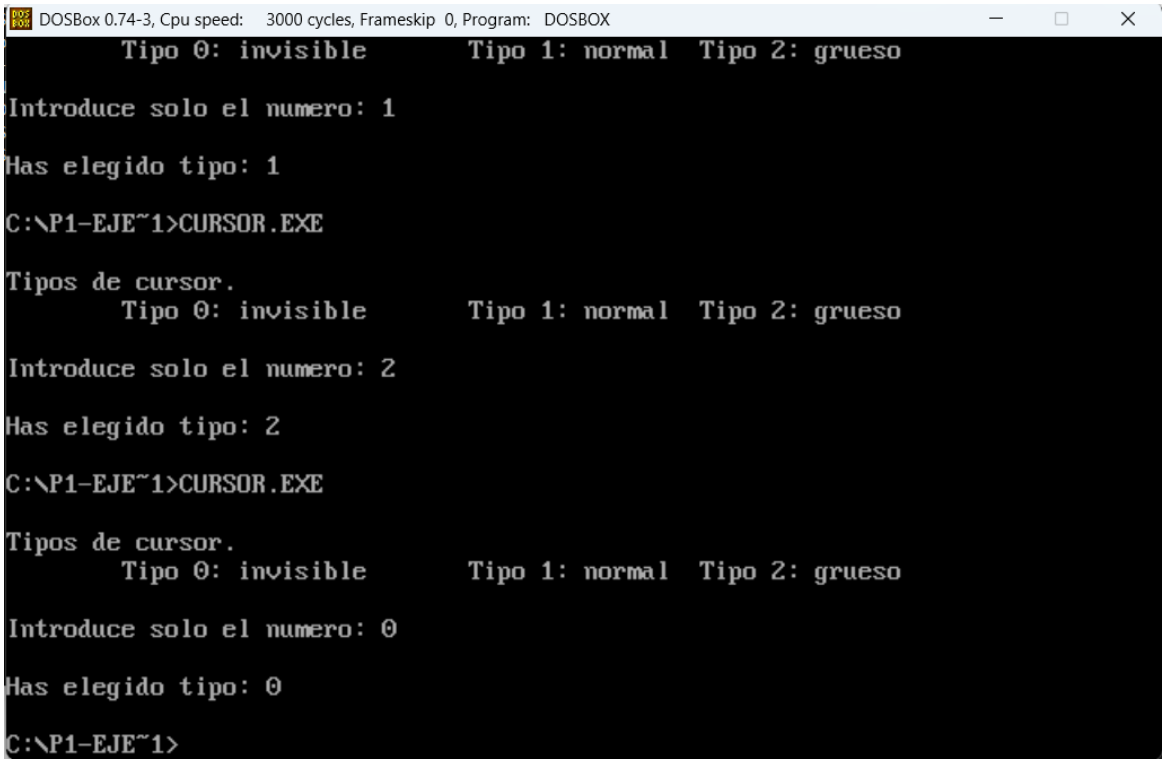

Figura 4: Ejecución de cursor invisible (tipo 0)

2.3. Ejercicio 3: *setvideomode()*

Objetivo: **fija el modo de video deseado**.

Para implementar esta función hay que usar la interrupción $10h$ de la BIOS, específicamente la función $00h$ (que se carga en el registro AH).

```c
void setvideomode(unsigned char modo){
    union REGS inregs, outregs;
    inregs.h.ah = 0x00;
    inregs.h.al = modo;
    int86(0x10,&inregs,&outregs);
    return;
}
```

Listing 7: Función setvideomode()

Un ejemplo de *main* para esta función es el siguiente:

```c
#define TEXTO 0x03
#define GRAFICO 0x04 // O 0x13 para 320x200 256 colores

int main(){
    printf("Modo grafico para dibujar");
    setvideomode(GRAFICO);  //0x04

    printf("Presiona una tecla para volver a modo texto...");
    mi_pausa();

    // Volver a modo texto antes de salir
    setvideomode(TEXTO); //0x03

    return 0;
}
```

Listing 8: Función principal para setvideomode()

Se han definido dos variables globales para cambiar los modos:

- *TEXTO*: Modo $AL = 03h$, con resolución 80x25, 16 colores.
- *GRAFICO*: Modo $AL = 04h$, con resolución 320x200, 4 colores.

Para ver el programa completo para probar el correcto funcionamiento de la función *setvideomode()* leer en el anexo 4.3.

2.4. Ejercicio 4: *getvideomode()*

Objetivo: **obtiene el modo de video actual**.

Para implementar esta función hay que consultar el estado actual del adaptador de vídeo. Según la documentación de la BIOS, esto se hace con la Interrupción $10h$, función $0Fh$.

```
unsigned char getvideomode(){
   union REGS inregs, outregs;
   inregs.h.ah = 0x0F; // Obtener modo de video actual
   int86(0x10,&inregs,&outregs);
   return outregs.h.al;
}
```

Listing 9: Función getvideomode()

Además, se hace uso de la función anterior [7] en el programa *main*.

```
int main(){
   unsigned char modo_original;
   modo_original = getvideomode();
   printf("Modo: %c\n", modo_original);

   printf("Cambiar a modo grafico para dibujar");
   setvideomode(GRAFICO);  // GRAFICO = 0x04
   mi_pausa();

   printf("Presiona una tecla para volver a modo original...");

   // Volver al modo original, sea cual sea
   setvideomode(modo_original);

   return 0;
}
```

Listing 10: Función principal para getvideomode()

Para ver el programa completo para probar el correcto funcionamiento de la función *getvideomode()* leer en el anexo 4.4.

2.5. Ejercicio 5: *textcolor()*

Objetivo: **modifica el color de primer plano con que se mostrarán los caracteres**.

Para escribir un carácter en pantalla se usa la interrupción $10h$, con el número de función $9 = AH$.
No tiene salida. Como parámetros de entrada tenemos:

- AH = 9
- AL = código ASCII del carácter
- BL = color
    - el primer cuarteto fija el color de fondo

- el segundo cuarteto el color del carácter

- BH = 0
- CX = número de repeticiones

Se necesitan variables que guarden los colores deseados para que todas las funciones de escritura lo sepan, como por ejemplo:

```
unsigned char colorActualTexto = 7; // Gris claro
```

Listing 11: Variable global

Luego hay que usar esas variables dentro de la función *cputchar()* para imprimir el caracter.

Para este ejercicio solo se muestra la implementación de la función que cambia el color del texto ([12]).

```
unsigned char colorActualTexto = 7; // Gris claro

void textcolor(int nuevo_color) {
    colorActualTexto = nuevo_color;
}
```

Listing 12: Función textcolor

Para ver un ejemplo completo en el que se usan las funciones *textcolor()*, *textbackground()* y *cputchar()* ver anexo 4.5.

2.6. Ejercicio 6: *textbackground()*

Objetivo: **modifica el color de fondo con que se mostrarán los caracteres**.

Es importante mencionar que **los colores de fondo sólo pueden tomar valores entre el 0 y el 7**. Esto se menciona en el documento de la práctica proporcionado por el profesor.

Una opción para usar las dos funciones anteriores (*textcolor()* y *textbackground()*) en una sola sería la propuesta por el profesor:

```
#include <stdio.h>
#include <dos.h>

unsigned char cfondo;
unsigned char ctexto;

void escribir_char_con_color(char c){
    union REGS inregs, outregs;
    inregs.h.ah = 0x09; // Funcion escribir caracter
    inregs.h.al = c;    //una funcion mas general debe recibir el caracter a imprimir
    inregs.h.bl = cfondo << 4 | ctexto; // Fusion de color de texto y fondo en un
        solo byte
    inregs.h.bh = 0x00;
    inregs.x.cx = 1; // Numero de veces que se repite el caracter
    int86(0x10,&inregs,&outregs);
    return;
}

int main(){
    printf("\nc.fondo=rojo, c.texto=azul => ");
    cfondo=4; ctexto=1; // cfondo=4=rojo , ctexto=1=azul
    escribir_char_con_color('H');

    printf("\nc.fondo=verde, c.texto=negro => ");
    cfondo=2; ctexto=0; // cfondo=2=verde , ctexto=0=negro
    escribir_char_con_color('O');

```

```
    printf("\nc.fondo=gris␣claro,␣c.texto=cian␣=>␣");
    cfondo=7; ctexto=3; // cfondo=7=gris claro , ctexto=13=cian
    escribir_char_con_color('L');

    printf("\nc.fondo=magenta,␣c.texto=verde␣claro␣=>␣");
    cfondo=5; ctexto=10; // cfondo=5=magenta , ctexto=10=verde claro
    escribir_char_con_color('A');

    return 0;
}
```

Listing 13: Programa proporcionado por el profesor

Ejecución para ver la palabra 'HOLA' con distintos colores [9]:

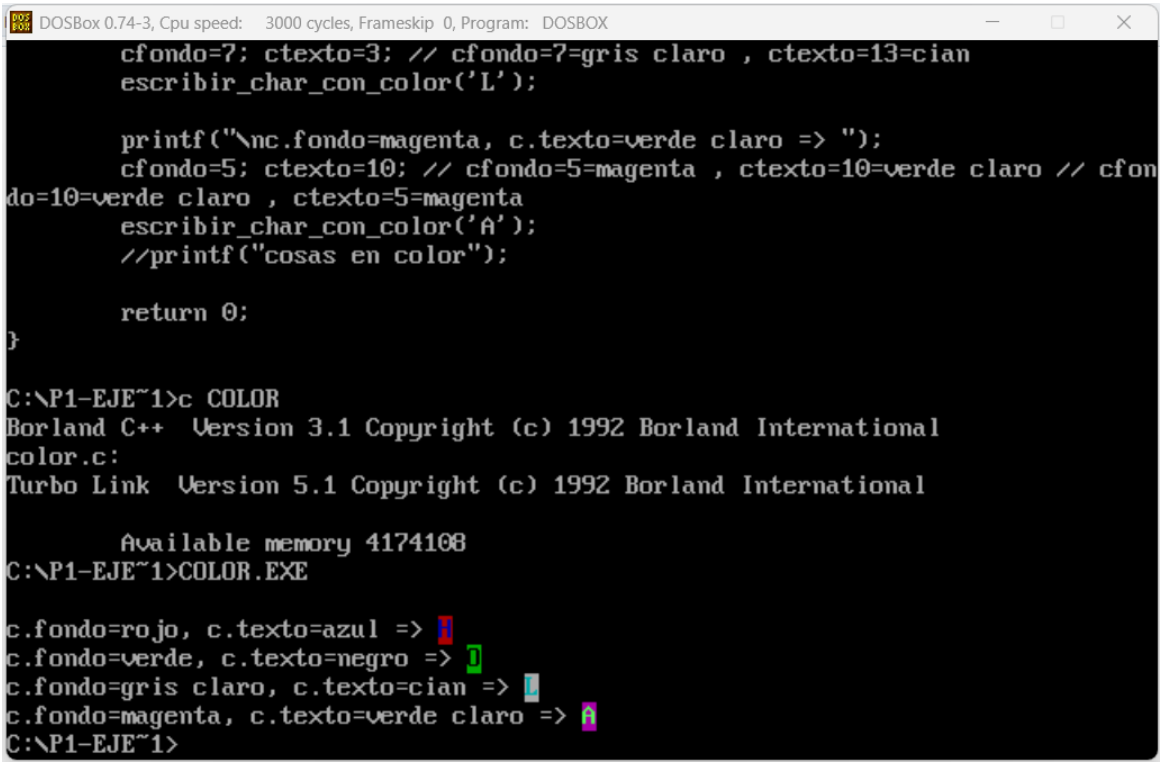

Figura 5: Ejemplo de color de texto y de fondo con palabra 'HOLA'

Otra opción sería definir una función para cambiar el color de fondo y combinarla con la función *textcolor()* ([12]) y con la función *cputchar()* del ejercicio 8 ([16]).

```
unsigned char colorActualFondo = 7; // Gris claro

void textbackground(int nuevo_color) {
    colorActualFondo = nuevo_color;
}
```

Listing 14: Función textbackground

Para ver un ejemplo completo en el que se usan las funciones *textcolor()*, *textbackground()* y *cputchar()* ver anexo 4.5.

2.7. Ejercicio 7: *clrscr()*

Objetivo: **borra toda la pantalla**.

En clase se propusieron varias ideas:

- Dar 25 retornos de carro con `printf("\n\n...\n");`. Al imprimir 25 saltos de línea se empuja el texto anterior hacia arriba hasta que desaparece de la ventana.

  La desventaja es que el cursor se quedaría en la parte de abajo de la pantalla, en vez de posicionarlo arriba como sería deseable. Si se usase este método se podría hacer uso de la función *gotoxy(0,0)*[3] para poner el cursor arriba del todo (en la coordenada 0,0).

- Llamar al modo de video usando la función *setvideomode(3)*[7] que provoca un parpadeo negro porque reinicia la tarjeta gráfica.

  La desventaja es que se pierde la configuración de colores que se hubiese establecido previamente.

- Usar la interrupción $10h$ y función $06h$ que se explica en el guión de la práctica para hacer *scroll vertical*.

La más acertada sería la tercera idea, usando la BIOS. Se borrará un recuadro que va desde la fila 0, columna 0, hasta la fila 24, columna 79 porque la pantalla en modo texto tiene 80 columnas y 25 filas.

La función número 6 tiene de entrada:

- AH = 6
- AL = número de líneas a desplazar. Si $AL = 0$ entonces no se desplaza, sino que borra esa zona.
- BH = color para los espacios en blanco
- CH = línea de la esquina superior izquierda
- CL = columna de la esquina superior izquierda
- DH = línea de la esquina inferior derecha
- DL = columna de la esquina inferior derecha

Entonces la implementación de la función *clrscr()* podría ser:

```
unsigned char cfondo=0; // Negro
unsigned char ctexto=15; // Blanco

void clrscr() {
    union REGS inregs, outregs;

    inregs.h.ah = 0x06;      // Funcion 6: Desplazamiento hacia arriba (Scroll)
    inregs.h.al = 0x00;      // AL = 0 indica borrar toda la zona seleccionada

    // El color con el que se rellena la pantalla vacia
    inregs.h.bh = (cfondo << 4) | ctexto;

    inregs.h.ch = 0;         // Fila de la esquina superior izquierda
    inregs.h.cl = 0;         // Columna de la esquina superior izquierda
    inregs.h.dh = 24;        // Fila de la esquina inferior derecha (25 filas en
        total)
    inregs.h.dl = 79;        // Columna de la esquina inferior derecha (80 columnas
        en total)

    int86(0x10, &inregs, &outregs); // Interrupcion de video
}
```

Listing 15: Función clrscr

Para ver el código del ejemplo completo ver anexo 4.6.

En la imagen se observa la compilación y ejecución[6]. El programa se queda esperando a que el usuario pulse una tecla.

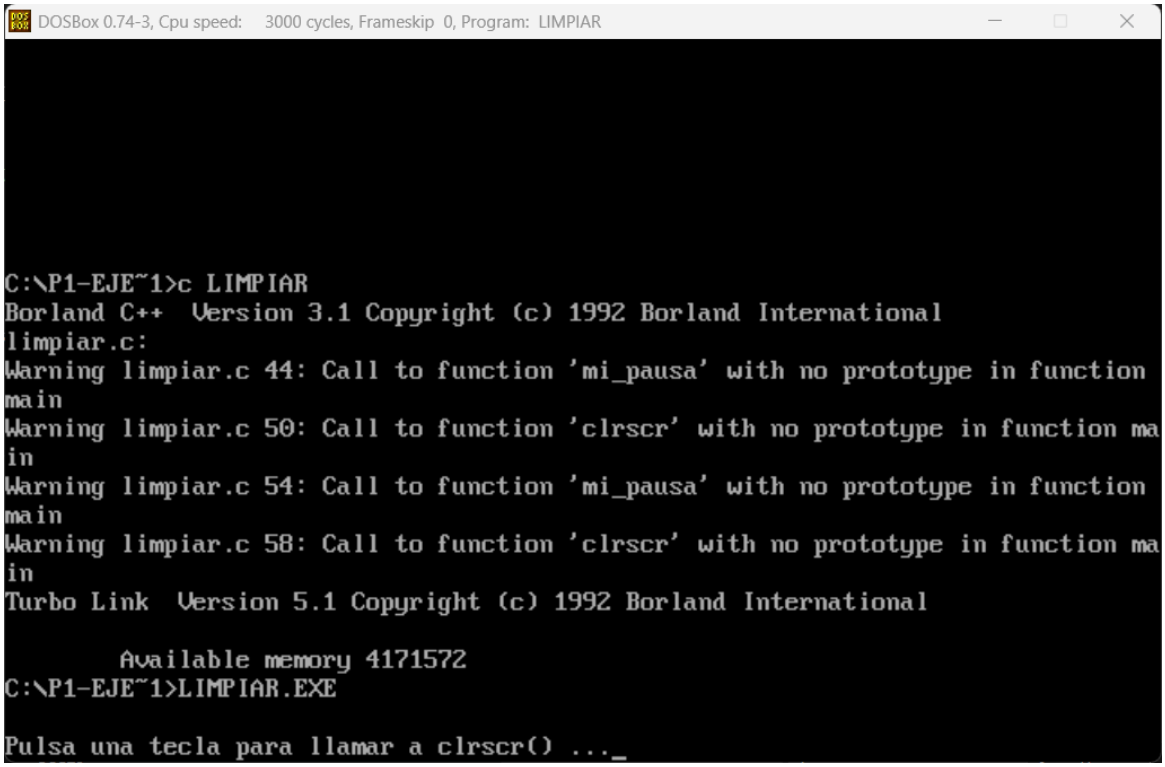

Figura 6: Ejemplo de compilación y ejecución de clrscr()

Justo antes de borrar se cambia el color de fondo a azul[7]. Así se ve claramente cómo la función de la BIOS pinta todo el recuadro de la pantalla (las 80 columnas y 25 filas) de ese color de una sola vez.

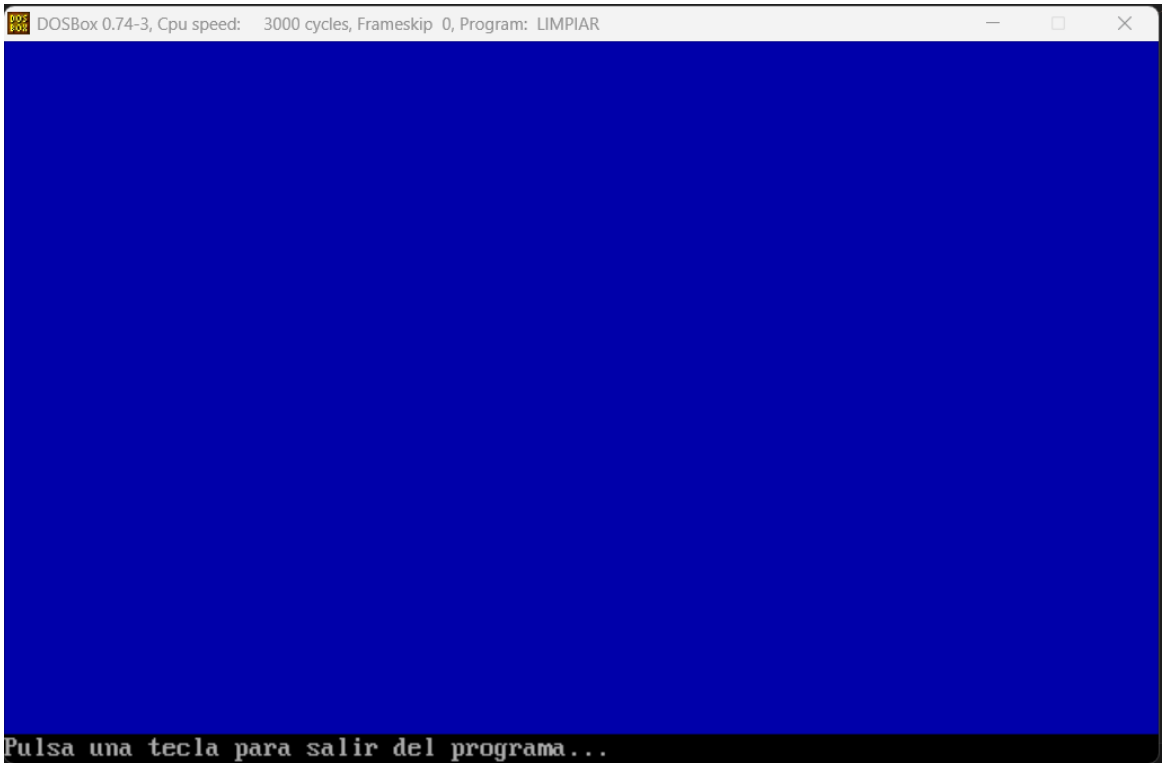

Figura 7: Ejemplo con la pantalla azul

Al pulsar una tecla para salir se reestablece la pantalla con fondo negro[8].

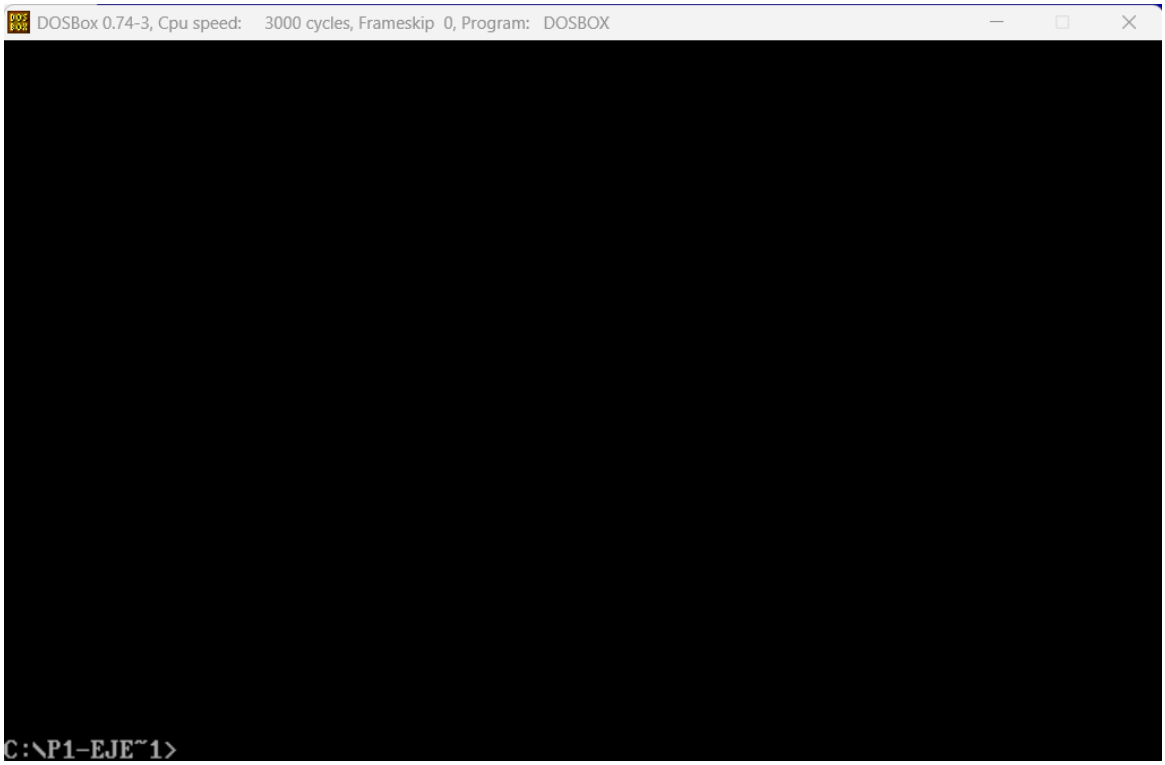

Figura 8: Ejemplo de pantalla 'limpia'

2.8. Ejercicio 8: *cputchar()*

Objetivo: **escribe un carácter en pantalla con el color indicado actualmente**.

Esta es la función que completa la funcionalidad de cambiar el color del texto y el color del fondo del texto que se solicita en los ejercicios 5 y 6.

La propuesta de la función es la siguiente:

```
unsigned char colorActualTexto = 4; // Rojo
unsigned char colorActualFondo = 7; // Gris claro

void cputchar(char c){
    union REGS inregs, outregs;
    inregs.h.ah = 0x09; // Funcion 9: escribir caracter y atributo
    inregs.h.al = c;    // Caracter a imprimir
    inregs.h.bl = (colorActualFondo << 4) | colorActualTexto; // Fusion de color de
        texto y fondo
    inregs.h.bh = 0x00; // Pagina de video
    inregs.x.cx = 1;    // Numero de veces que se repite el caracter
    int86(0x10, &inregs, &outregs);
}
```

Listing 16: Función cputchar()

Entonces, combinando las funciones *textcolor()* y *textbackground()* se puede implementar el siguiente programa:

```
int main(){
    printf("\nc.fondo=rojo, c.texto=azul => ");
    colorActualTexto = 1; colorActualFondo = 4; // Texto Azul; Fondo Rojo
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('P');

    printf("\nc.fondo=gris claro, c.texto=cian => ");
    colorActualTexto = 3; colorActualFondo = 7; // Texto Cian; Fondo Gris claro
```

```
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('D');

    printf("\nc.fondo=magenta,␣c.texto=verde␣claro␣=>␣");
    colorActualTexto = 10; colorActualFondo = 5; // Texto verde claro; Fondo Magenta
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('I');

    printf("\nc.fondo=verde,␣c.texto=negro␣=>␣");
    colorActualTexto = 0; colorActualFondo = 2; // Texto negro; Fondo Verde
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('H');

    return 0;
}
```

Listing 17: Función principal

Para ver el ejemplo completo en el que se usan las funciones *textcolor()*, *textbackground()* y *cputchar()* ver anexo 4.5.

En la siguiente imagen ([9]) podemos observar las dos soluciones propuestas, la del profesor con la palabra 'HOLA' y la segunda propuesta con la palabra 'PDIH'.

Figura 9: Ejemplo de color de texto y de fondo con palabra 'PDIH'

2.9. Ejercicio 9: *getche()*

Objetivo: **obtiene un carácter de teclado y lo muestra en pantalla**.

Para gestionar el teclado se usa la interrupción de la BIOS $16h$. La subfunción 1 detecta la pulsación de una tecla. La entrada es $AH = 1$. La salida es 0 si hay una tecla en el búfer, y 1 si el búfer está vacío.

```
void getche(){
    union REGS inregs, outregs;
    int caracter;
```

```
    // Solicitar
    printf("\nPulsa una tecla... ");
    inregs.h.ah = 1;
    int86(0x21, &inregs, &outregs);

    caracter = outregs.h.al;

    // Mostrar
    printf("\nHas pulsado: ");
    inregs.h.ah = 2;
    inregs.h.dl = caracter;
    int86(0x21, &inregs, &outregs);
}
```

Listing 18: Función getche()

Para ver el programa completo para probar el correcto funcionamiento de la función *getvideomode()* leer en el anexo 4.7.

Ejemplo al pulsar la tecla 'a':

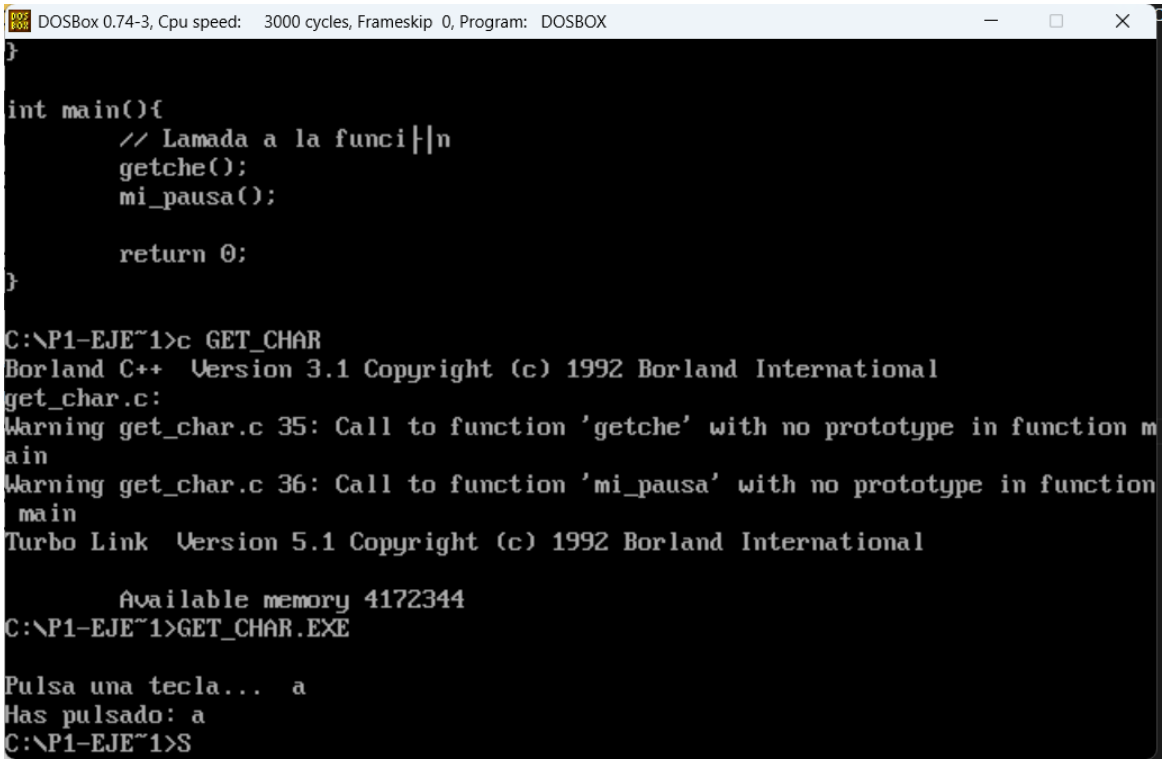

Figura 10: Ejemplo de getche()

2.10. Ejercicio 10: *pixel()*

Objetivo: **dibujar un pixel en modo gráfico** (la función recibirá la coordenada x, y y el color del punto).

Se debe trabajar en modo gráfico, por eso hay que usar la función *setvideomode()* ([7]) del ejercicio 3.

La función *pixel()* sería:

```
#define BYTE unsigned char

void pixel(int x, int y, BYTE C){
    union REGS inregs, outregs;

```

```
    inregs.x.cx = x;        // Columna (coordenada X)
    inregs.x.dx = y;        // Fila (coordenada Y)
    inregs.h.al = C;        // Color del pixel
    inregs.h.ah = 0x0C;     // Funci n 0Ch: iluminar un pixel en modo gr fico

    int86(0x10, &inregs, &outregs); // Llamada a la interrupci n BIOS para pantalla
}
```

Listing 19: Función pixel

Para probar que funciona correctamente se usa el siguiente main:

```
int main(){
    // Cambiar a modo gr fico gr fico (320x200 con 4 colores)
    setvideomode(4);

    // Pintar algunos p xeles (formando una l nea diagonal)
    pixel(01, 40, 7); // Color 7: gris claro
    pixel(10, 50, 1); // Color 1: azul
    pixel(15, 60, 2); // Color 2: verde
    pixel(20, 70, 3); // Color 3: cyan
    pixel(25, 80, 7); // Color 7
    pixel(30, 90, 1); // Color 1
    pixel(35, 100, 2); // Color 2
    pixel(40, 110, 3); // Color 3

    pixel(45, 100, 7); // Color 7
    pixel(50, 90, 1); // Color 1
    pixel(55, 80, 2); // Color 2
    pixel(60, 70, 3); // Color 3
    pixel(65, 60, 7); // Color 7
    pixel(70, 50, 1); // Color 1
    pixel(75, 40, 2); // Color 2

    // Hacer una pausa para ver el resultado
    mi_pausa();

    // Restaurar el modo texto a color
    setvideomode(3);

    return 0;
}
```

Listing 20: Función main para pixel

Los pixeles pintados forman una V, empezando por la coordenada $(1, 40)$ con el color gris claro, con vértice color cyan en la coordenada $(40, 110)$, y teminando en la coordenada $(75, 40)$ con color verde.

Para ver el programa completo para probar el correcto funcionamiento de la función *pixel* leer en el anexo: 4.8

Ejecución del ejemplo en forma de V[11]:

Figura 11: Ejemplo de la función pixel

3. Requisitos ampliados

3.1. Ejercicio 11

Implementar una función que permita dibujar un recuadro en la pantalla en modo texto. Recibirá como parámetros las coordenadas superior izquierda e inferior derecha del recuadro, el color de primer plano y el color de fondo.

3.2. Ejercicio 12

Implementar en lenguaje C un programa que establezca modo gráfico CGA (modo=4) para crear dibujos sencillos en pantalla.

3.3. Ejercicio 13

Implementar un programa sencillo que realice un dibujo sencillo de tipo "ascii art".

4. Anexo

4.1. Ejemplo completo para la función *gotoxy()*

```
#include <stdio.h>
#include <dos.h>

// Ejercicio P1:
// Coloca el cursor en una posici n determinada dada por el usuario

void mi_pausa(){
```

```
    union REGS inregs, outregs;
    inregs.h.ah = 8;
    int86(0x21, &inregs, &outregs);
}

void xy(int x, int y){
    union REGS inregs, outregs;
    inregs.h.ah = 0x02;
    inregs.h.bh = 0x00;
    inregs.h.dh = y;
    inregs.h.dl = x;
    int86(0x10,&inregs,&outregs);
    return;
}

int main(){
    int pos_x, pos_y; //Guardar posici n determinada

    // Solicitar coordenadas
    printf("Introduce␣la␣posicion␣x␣(columna)");
    scanf("%d", &pos_x);
    printf("Introduce␣la␣posicion␣y␣(fila)");
    scanf("%d", &pos_y);

    xy(pos_x,pos_y); // Llamada a la funci n

    printf("*"); // Imprimir asterisco en la posici n

    mi_pausa();

    return 0;
}
```

4.2. Ejemplo completo para la función *setcursortype()*

```
#include <stdio.h>
#include <dos.h>

// Ejercicio P1: setcursortype()
// Fijar el aspecto del cursor, debe admitir tres valores: INVISIBLE, NORMAL y GRUESO
    .


void mi_pausa(){
   union REGS inregs, outregs;
     inregs.h.ah = 8;
     int86(0x21, &inregs, &outregs);
}

void setcursortype(int tipo_cursor){
    union REGS inregs, outregs;
    inregs.h.ah = 0x01;
    switch(tipo_cursor){
        case 0: //invisible
            inregs.h.ch = 010;
            inregs.h.cl = 000;
            break;
        case 1: //normal
            inregs.h.ch = 010;
            inregs.h.cl = 010;
            break;
        case 2: //grueso
```

```
            inregs.h.ch = 000;
            inregs.h.cl = 010;
            break;

    }
    int86(0x10, &inregs, &outregs);
}

int main(){
    int tmp;

    printf("\nTipos de cursor. \n");
    printf("\tTipo 0: invisible \tTipo 1: normal \tTipo 2: grueso \n\nIntroduce solo
        el numero: ");
    scanf("%d", &tmp); // Leer de teclado

    printf("\nHas elegido tipo: %d\n", tmp); //Mostrar elecci n

    setcursortype(tmp);
    mi_pausa();

    return 0;
}
```

4.3. Ejemplo completo para la función *setvideomode()*

```
#include <stdio.h>
#include <dos.h>

#define TEXTO 0x03
#define GRAFICO 0x04 // O 0x13 para 320x200 256 colores

void setvideomode(unsigned char modo){
    union REGS inregs, outregs;
    inregs.h.ah = 0x00;
    inregs.h.al = modo;
    int86(0x10,&inregs,&outregs);
    return;
}

int main(){

    printf("Modo grafico para dibujar");
    setvideomode(GRAFICO); //0x04

    printf("Presiona una tecla para volver a modo texto...");
    mi_pausa();

    // Volver a modo texto antes de salir
    setvideomode(TEXTO); //0x03

    return 0;
}
```

4.4. Ejemplo completo para la función *getvideomode()*

```
#include <stdio.h>
#include <dos.h>

#define GRAFICO 0x04

```

```
void mi_pausa(){
   union REGS inregs, outregs;
     inregs.h.ah = 8;
     int86(0x21, &inregs, &outregs);
}

void setvideomode(unsigned char modo){
    union REGS inregs, outregs;
    inregs.h.ah = 0x00; // Para cambiar el modo de pantalla
    inregs.h.al = modo; // Indicar a qu  modo espec fico cambiar
    int86(0x10,&inregs,&outregs);
    return;
}

unsigned char getvideomode(){
    union REGS inregs, outregs;
   inregs.h.ah = 0x0F; // Obtener modo de video actual
   int86(0x10,&inregs,&outregs);
   return outregs.h.al;
}

int main(){
   unsigned char modo_original;
   modo_original = getvideomode();
   printf("Modo: %c\n", modo_original);

   printf("Cambiar a modo gr fico para dibujar");
    setvideomode(GRAFICO);  // GRAFICO = 0x04
   mi_pausa();

    printf("Presiona una tecla para volver a modo original...");

   // Volver al modo original, sea cual sea
   setvideomode(modo_original);

    return 0;
}
```

4.5. Ejemplo completo para las funciones *textcolor()*, *textbackground()* y *cputchar()*

```
#include <stdio.h>
#include <dos.h>

unsigned char colorActualTexto = 4; // Rojo
unsigned char colorActualFondo = 7; // Gris claro

// Funcion para cambiar el color del texto
void textcolor(int nuevo_color) { colorActualTexto = nuevo_color;}

// Funcion para cambiar el color de fondo
void textbackground(int nuevo_color) { colorActualFondo = nuevo_color;}

// Funcion para escribir el car cter
void cputchar(char c){
    union REGS inregs, outregs;
    inregs.h.ah = 0x09; // Funci n 9: escribir caracter y atributo
    inregs.h.al = c;    // Caracter a imprimir
    inregs.h.bl = (colorActualFondo << 4) | colorActualTexto; // Fusi n
    inregs.h.bh = 0x00;
    inregs.x.cx = 1;
    int86(0x10, &inregs, &outregs);
```

```
}

int main(){
    printf("\nc.fondo=rojo,␣c.texto=azul␣=>␣");
    colorActualTexto = 1; colorActualFondo = 4; // Texto Azul; Fondo Rojo
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('P');

    printf("\nc.fondo=gris␣claro,␣c.texto=cian␣=>␣");
    colorActualTexto = 3; colorActualFondo = 7; // Texto Cian; Fondo Gris claro
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('D');

    printf("\nc.fondo=magenta,␣c.texto=verde␣claro␣=>␣");
    colorActualTexto = 10; colorActualFondo = 5; // Texto verde claro; Fondo Magenta
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('I');

    printf("\nc.fondo=verde,␣c.texto=negro␣=>␣");
    colorActualTexto = 0; colorActualFondo = 2; // Texto negro; Fondo Verde
    textbackground(colorActualFondo); textcolor(colorActualTexto);
    cputchar('H');

    return 0;
}
```

4.6. Ejemplo completo para la función *clrscr()*

```
#include <stdio.h>
#include <dos.h>

unsigned char cfondo=0; // Negro
unsigned char ctexto=15; // Blanco

void mi_pausa(){
    union REGS inregs, outregs;
    inregs.h.ah = 8;          // Funci n 8: leer tecla sin mostrarla
    int86(0x21, &inregs, &outregs);
}

//Funci n para posicionar el cursor en coordenadas (x, y)
void xy(int x, int y){
    union REGS inregs, outregs;
    inregs.h.ah = 0x02;
    inregs.h.bh = 0x00;
    inregs.h.dh = y;
    inregs.h.dl = x;
    int86(0x10, &inregs, &outregs);
    return;
}

void clrscr() {
    union REGS inregs, outregs;

    inregs.h.ah = 0x06;       // Funcion 6: Desplazamiento hacia arriba (Scroll)
    inregs.h.al = 0x00;       // AL = 0 indica borrar toda la zona seleccionada

    // El color con el que se rellena la pantalla vacia
    inregs.h.bh = (cfondo << 4) | ctexto;

    inregs.h.ch = 0;          // Fila de la esquina superior izquierda
    inregs.h.cl = 0;          // Columna de la esquina superior izquierda
```

```
    inregs.h.dh = 24;        // Fila de la esquina inferior derecha (25 filas en total)
    inregs.h.dl = 79;        // Columna de la esquina inferior derecha (80 columnas en total)

    int86(0x10, &inregs, &outregs); // Interrupcion de video
}

int main() {
    printf("\nPulsa una tecla para llamar a clrscr() ...");
    mi_pausa();

    // Cambiar el color de fondo a AZUL justo antes de borrar
    cfondo = 1; //Azul
    ctexto = 15; //Blanco

    clrscr();

    printf("\nPulsa una tecla para salir del programa...");

    mi_pausa();

    // Restaurar el fondo a negro
    cfondo = 0; //Negro
    clrscr();

    return 0;
}
```

4.7. Ejemplo completo para la función *getche*

```
#include <stdio.h>
#include <dos.h>

// Ejercicio P1:
// Obtiene un car cter de teclado y lo muestra en pantalla


void mi_pausa(){
   union REGS inregs, outregs;
     inregs.h.ah = 8;
     int86(0x21, &inregs, &outregs);
}

void getche(){
    union REGS inregs, outregs;
    int caracter;

    // Solicitar
    printf("\nPulsa una tecla...  ");
    inregs.h.ah = 1;
    int86(0x21, &inregs, &outregs);

    caracter = outregs.h.al;

    // Mostrar
    printf("\nHas pulsado: ");
    inregs.h.ah = 2;
    inregs.h.dl = caracter;
    int86(0x21, &inregs, &outregs);

}
```

```

int main(){
    // Lamada a la funci n
    getche();
    mi_pausa();

    return 0;
}
```

4.8. Ejemplo completo para la función *pixel()*

```
#include <stdio.h>
#include <dos.h>

#define BYTE unsigned char

void mi_pausa(){
   union REGS inregs, outregs;
   inregs.h.ah = 8;
   int86(0x21, &inregs, &outregs);
}

void setvideomode(unsigned char modo){
    union REGS inregs, outregs;
    inregs.h.ah = 0x00;
    inregs.h.al = modo;
    int86(0x10,&inregs,&outregs);
    return;
}

void pixel(int x, int y, BYTE C){
    union REGS inregs, outregs;

    inregs.x.cx = x;        // Columna (coordenada X)
    inregs.x.dx = y;        // Fila (coordenada Y)
    inregs.h.al = C;        // Color del pixel
    inregs.h.ah = 0x0C;     // Funci n 0Ch: iluminar un pixel en modo gr fico

    int86(0x10, &inregs, &outregs); // Llamada a la interrupci n BIOS para pantalla
}

int main(){
    // Cambiar a modo gr fico gr fico (320x200 con 4 colores)
    setvideomode(4);

    // Pintar algunos p xeles (formando una v)
    pixel(01, 40, 7); // Color 7: gris claro
    pixel(10, 50, 1); // Color 1: azul
    pixel(15, 60, 2); // Color 2: verde
    pixel(20, 70, 3); // Color 3: cyan
    pixel(25, 80, 7);   pixel(30, 90, 1);
    pixel(35, 100, 2);  pixel(40, 110, 3);
    pixel(45, 100, 7);  pixel(50, 90, 1);
    pixel(55, 80, 2);   pixel(60, 70, 3);
    pixel(65, 60, 7);   pixel(70, 50, 1);   pixel(75, 40, 2);

    // Hacer una pausa para ver el resultado
    mi_pausa();

    // Restaurar el modo texto a color
    setvideomode(3);

```

```
    return 0;
}
```

4.9. Código completo para el ejercicio 11

4.10. Código completo para el ejercicio 12

4.11. Código completo para el ejercicio 13